

PA0 — Environment Setup & Chat Application

CS 158A Computer Networks

The goal of PA0 is to get your development environment working so that PA1, PA2, and PA3 can focus entirely on network concepts. By the end of this assignment you will have Python installed, a virtual environment configured, PyCharm set up, and a working TCP chat application running across two terminals on your machine.

Part 1 — Install Python

You need Python 3.9 or higher. To check if Python is already installed, open a terminal and run:

```
python --version  
# or  
python3 --version
```

If you see Python 3.x.x you can move on to Part 2. If not, follow the steps for your operating system below.

Windows

1. Go to <https://www.python.org/downloads/> and download the latest Python 3 installer.
2. Run the installer. On the first screen, check the box that says "Add Python to PATH" before clicking Install. This is easy to miss and causes problems later.
3. Once the install finishes, open Command Prompt (search for cmd in the Start menu) and run:

```
python --version
```

You should see something like Python 3.12.2.

macOS

4. Go to <https://www.python.org/downloads/> and download the macOS installer.
5. Run the .pkg file and follow the prompts.

6. Open Terminal (Cmd + Space, then type Terminal) and run:

```
python3 --version
```

⚠ On macOS, always use python3 and pip3 rather than python and pip, unless you are inside a virtual environment (covered below).

Linux (Ubuntu / Debian)

Python 3 is usually pre-installed. Check with:

```
python3 --version
```

If it is missing or below 3.9, run:

```
sudo apt update  
sudo apt install python3 python3-pip python3-venv
```

Part 2 — Set Up a Virtual Environment

A virtual environment is an isolated Python installation for your project. It keeps your project's dependencies separate from the rest of your system, which avoids version conflicts across different assignments.

Step 1 — Create a folder for PA0

Pick a location on your machine and create a folder. From the terminal:

```
mkdir pa0_chat  
cd pa0_chat
```

Step 2 — Create the virtual environment

Windows:

```
python -m venv venv
```

macOS / Linux:

```
python3 -m venv venv
```

This creates a folder called venv inside pa0_chat. You only do this once.

Step 3 — Activate the virtual environment

You need to activate the environment every time you open a new terminal to work on this project.

Windows (Command Prompt):

```
venv\Scripts\activate
```

Windows (PowerShell):

```
venv\Scripts\Activate.ps1
```

⚠ If you get a permissions error in PowerShell, run: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` and try again.

macOS / Linux:

```
source venv/bin/activate
```

Once activated, your terminal prompt will show (venv) at the beginning:

```
(venv) C:\Users\yourname\pa0_chat>
```

Step 4 — Confirm it is working

```
python --version
```

You should see Python 3.x.x. PA0 uses only the Python standard library (socket and threading), so there are no packages to install. Future assignments may require packages installed with pip install <package-name> while the environment is active.

Part 3 — Install PyCharm

7. Go to <https://www.jetbrains.com/pycharm/download/>
8. Download the Community Edition (free).
9. Install it using the default options.

Open the PA0 Project in PyCharm

10. Launch PyCharm.
11. Click Open and navigate to your pa0_chat folder. Select it and click OK.
12. PyCharm will detect your virtual environment automatically. If it asks you to configure an interpreter, click Add Interpreter > Existing Environment and point it to venv/bin/python (macOS/Linux) or venv\Scripts\python.exe (Windows).

You should now see the project files in the left panel.

Part 4 — Download the PA0 Files

Download the following two files and place them inside your pa0_chat folder:

- server.py
- client.py

After placing them, your folder structure should look like this:

```
pa0_chat/  
├── venv/  
├── server.py  
└── client.py
```

Part 5 — Run the Chat Application

In this assignment you will test a simple TCP based client/server chat application. You will learn more about network sockets and applications in Chapter 2, but here is the gist – a TCP chat application sends text messages between two programs over a reliable network connection. The server starts first, opens a port, and waits for someone to connect. The client initiates the connection to the server. Once connected, both sides can send and receive messages freely until either party disconnects.

You will use two separate terminal windows for testing PA0. Think of one terminal as one person's computer and the other as another person's computer. In real life these would be two different machines on a network — here, both run on your machine using the loopback address 127.0.0.1 (localhost).

Terminal 1 — Start the Server

Open a terminal, navigate to your `pa0_chat` folder, and activate the virtual environment.

Windows:

```
cd pa0_chat
venv\Scripts\activate
python server.py
```

macOS / Linux:

```
cd pa0_chat
source venv/bin/activate
python server.py
```

You should see:

```
[Server] Listening on 127.0.0.1:12345 ...
[Server] Waiting for a client to connect...
```

The server is now waiting. Leave this terminal open and open a second one.

Terminal 2 — Start the Client

Open a second terminal window, navigate to the same folder, activate the virtual environment again, and run:

Windows:

```
cd pa0_chat
venv\Scripts\activate
python client.py
```

macOS / Linux:

```
cd pa0_chat
source venv/bin/activate
python client.py
```

Part 6 — Expected Output

Once the client connects, both terminals will display a chat prompt.

Terminal 1 (Server)

```
[Server] Listening on 127.0.0.1:12345 ...
[Server] Waiting for a client to connect...
[Server] Client connected from ('127.0.0.1', 54321)
=====
Chat session started. Type a message and press Enter.
Type 'quit' to end the session.
=====

You:
```

Terminal 2 (Client)

```
[Client] Connected to server at 127.0.0.1:12345
=====
Chat session started. Type a message and press Enter.
Type 'quit' to end the session.
=====

You:
```

Sending Messages

Type a message in the client terminal and press Enter:

```
You: Hello from the client!
```

The server terminal will display:

```
[Client]: Hello from the client!
You:
```

Reply from the server terminal:

```
You: Hello back from the server!
```

The client terminal will display:

```
[Server]: Hello back from the server!  
You:
```

Both sides can type at any time. To end the session, type quit in either terminal and press Enter.

Part 7 — Troubleshooting

"Address already in use" / "Port already in use"

Something else is using port 12345 — possibly a previous run of server.py that didn't close cleanly.

- Wait 30 seconds and try again. The OS will release the port.
- Open server.py and client.py and change PORT = 12345 to another number like PORT = 12346. Make sure you change it in both files.

"Connection refused" when running client.py

The server is not running. Make sure Terminal 1 is open and server.py is showing the "Waiting for a client" message before you start client.py.

python command not found (macOS / Linux)

Use python3 instead, or make sure your virtual environment is activated — you should see (venv) in your prompt.

PowerShell execution policy error (Windows)

Run this once in PowerShell as Administrator:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Then try activating the virtual environment again.

PyCharm does not detect the virtual environment

Go to File > Settings > Project > Python Interpreter, click the gear icon, and select Add Interpreter > Existing Environment. Navigate to venv\Scripts\python.exe (Windows) or venv/bin/python (macOS/Linux).

Deliverable

Submit the following two items:

1. A screenshot showing both terminals side by side with at least two messages exchanged in each direction.
2. Two sentences confirming it worked and noting the port number you used.

That's it. The purpose of PA0 is setup, and the deliverable reflects that.

What This Sets You Up For

Every programming assignment in this course follows the same structure: sockets, a local server, and a client sending and receiving data. The concepts change — reliable transfer, DNS lookup, HTTP — but the tooling stays the same. Getting comfortable with this environment now means you can focus on the network concepts in PA1, PA2, and PA3 without fighting your setup.